DA-V — Distribuidor Aamplo · Video Distribution Service

> **Foco puro em backend**: arquitetura de distribuição de vídeo de alta performance sobre rede local (LAN) e remota (WAN), com qualidade de serviço (QoS) máxima e zero dependência de UI.

Visão Geral

O **DA-V** é um serviço de distribuição de vídeo em tempo real análogo às entranhas de produtos como AnyDesk e TeamViewer. O escopo deste projeto é **exclusivamente backend**: captura, compressão, transporte, controle de fluxo, reconexão e otimização adaptativa — tudo em C++ de alto desempenho com bindings opcionais em Python para automação e scripting.

O sistema evolui em três fases:

Fase	Escopo	Objetivo
Fase 1	LAN — distribuição local	Captura de tela + codificação + transporte UDP confiável
Fase 2	WAN — travessia NAT	Relay/TURN, ICE, hole-punching, tunelamento criptografado
Fase 3	Full-mesh + QoS Avançado	Multi-peer, adaptive bitrate, FEC, priorização de fluxo

Pilha Tecnológica

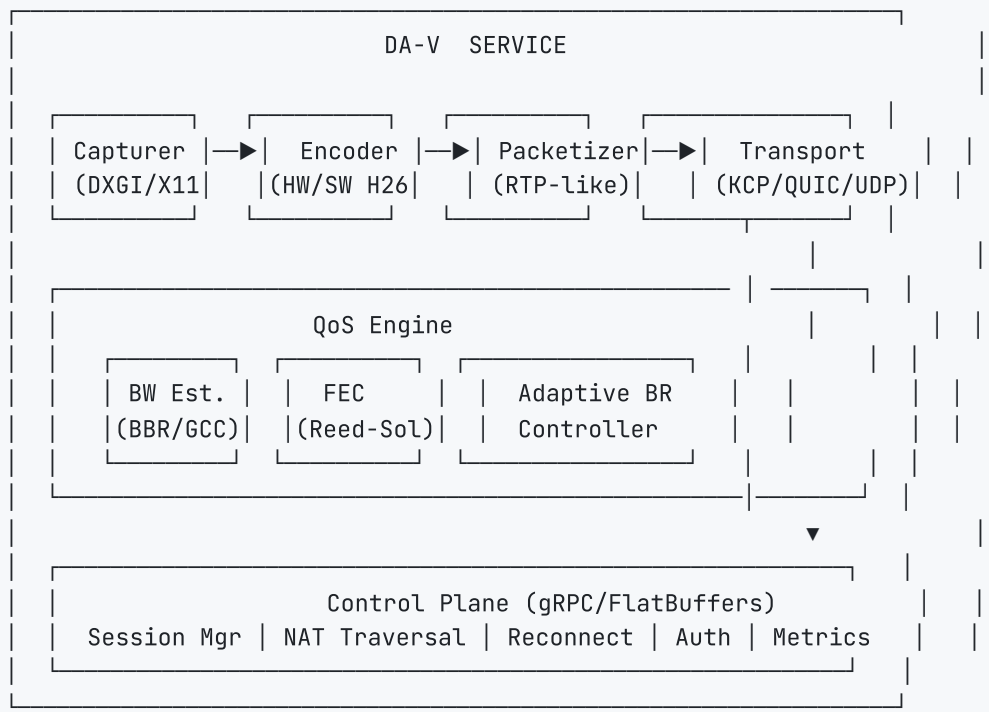
Linguagem Principal

- **C++23** — núcleo do serviço (captura, codec, rede, scheduler)
- **Python 3.12+** — harness de testes, scripts de benchmark e CLI de controle

Bibliotecas / Dependências

Camada	Biblioteca	Papel
Captura de tela	`DXGI Desktop Duplication API` (Win) / `X11 SHM` (Linux)	Captura GPU-side sem cópia desnecessária
Codec de vídeo	**FFmpeg 7.x** (libavcodec + libavformat) + **libyuv**	H.264 / H.265 / AV1 adaptativo
Hardware encode	**NVENC** (NVIDIA) / **QSV** (Intel) / **AMF** (AMD)	Zero-copy GPU encode
Rede	**libuv** (event loop) + **raw sockets** + **QUIC via libmsquic**	Transporte assíncrono
Protocolo confiável UDP	**KCP** (kcp.h)	ARQ de baixa latência sobre UDP
Criptografia	**libsodium** (ChaCha20-Poly1305 + X25519)	Canal seguro E2E
TLS (sinalização)	**mbedTLS** ou **OpenSSL 3.x**	Handshake de controle
Traversal NAT	**libnice** (ICE/STUN/TURN)	Hole-punching e relay
Serialização	**FlatBuffers**	Mensagens zero-copy de controle
Métricas / QoS	**prometheus-cpp** (embutido)	Exposição de métricas via /metrics
Logs	**spdlog**	Structured logging assíncrono
Build	**CMake 3.28 + vcpkg**	Gestão de dependências cross-platform
Testes	**GoogleTest + Catch2**	Unit e integration tests
Benchmark	**Google Benchmark**	Microbenchmarks de codecs e rede

Arquitetura de Alto Nível



Estrutura de Diretórios

```
Distribuidor_Aamplo-Video/
├── CMakeLists.txt                    # Root build
├── vcpkg.json                        # Dependências vcpkg
├── .clang-format                     # Estilo de código
├── .clang-tidy                       # Linting estático
│
├── src/
│   ├── core/                         # Núcleo do daemon
│   │   ├── main.cpp                  # Ponto de entrada
│   │   ├── daemon.hpp / .cpp         # Ciclo de vida do serviço
│   │   └── config.hpp / .cpp         # Configuração (TOML)
│   │
│   ├── capture/                      # Captura de tela
│   │   ├── capturer.hpp              # Interface abstrata
│   │   ├── dxgi_capturer.hpp/.cpp    # Windows DXGI Desktop Dup
│   │   ├── x11_capturer.hpp/.cpp     # Linux X11 SHM
│   │   └── frame_buffer.hpp          # Ring buffer de frames
│   │
│   ├── codec/                        # Codificação de vídeo
│   │   ├── encoder.hpp               # Interface abstrata
│   │   ├── nvenc_encoder.hpp/.cpp    # NVIDIA hardware encode
│   │   ├── qsv_encoder.hpp/.cpp      # Intel Quick Sync
│   │   ├── sw_encoder.hpp/.cpp       # Software fallback (x264/x265)
│   │   ├── frame_scaler.hpp/.cpp     # libyuv YUV conversão/escala
│   │   └── codec_params.hpp          # Parâmetros de codec
│   │
│   ├── packetizer/                   # Fragmentação e protocolo de frame
│   │   ├── rtp_packetizer.hpp/.cpp   # RTP-like framing (H.264 NAL)
│   │   ├── frame_assembler.hpp/.cpp  # Reassembly no receptor
│   │   └── packet.hpp                # Layout de pacote
│   │
│   ├── transport/                    # Camada de transporte
│   │   ├── transport.hpp             # Interface abstrata
│   │   ├── kcp_transport.hpp/.cpp    # KCP sobre UDP (ARQ)
│   │   ├── quic_transport.hpp/.cpp   # QUIC via libmsquic
│   │   ├── udp_socket.hpp/.cpp       # Raw UDP helpers
│   │   ├── socket_pool.hpp/.cpp      # Pool de conexões
│   │   └── bandwidth_estimator.hpp/.cpp # BBR / GCC estimador
│   │
│   ├── qos/                          # Motor de Qualidade de Serviço
│   │   ├── qos_engine.hpp/.cpp       # Orquestrador de QoS
│   │   ├── fec.hpp/.cpp              # Forward Error Correction (Reed-Solomon)
│   │   ├── adaptive_bitrate.hpp/.cpp # ABR controller (AIMD + ML hints)
│   │   ├── jitter_buffer.hpp/.cpp    # Jitter buffer no receptor
│   │   ├── packet_scheduler.hpp/.cpp # Priority scheduling (DSCP/QoS tags)
│   │   └── nack_handler.hpp/.cpp     # NACK / retransmissão seletiva
│   │
│   ├── crypto/                       # Criptografia e autenticação
│   │   ├── crypto_channel.hpp/.cpp   # ChaCha20-Poly1305 E2E
│   │   ├── key_exchange.hpp/.cpp     # X25519 ECDH
│   │   └── auth.hpp/.cpp             # Token-based auth (HMAC-SHA256)
│   │
│   ├── traversal/                    # Traversal de NAT
```

```
│   │   ├── ice_agent.hpp/.cpp        # ICE (libnice wrapper)
│   │   ├── stun_client.hpp/.cpp      # STUN queries
│   │   ├── turn_client.hpp/.cpp      # TURN relay
│   │   └── hole_punch.hpp/.cpp       # UDP hole punching
│   │
│   ├── session/                      # Gerenciamento de sessão
│   │   ├── session.hpp/.cpp          # Estado de sessão
│   │   ├── session_manager.hpp/.cpp  # Ciclo de vida multi-sessão
│   │   ├── reconnect.hpp/.cpp        # Lógica de reconexão (backoff exp.)
│   │   └── peer_registry.hpp/.cpp    # Registro de peers (LAN mDNS + manual)
│   │
│   ├── control/                      # Plano de controle
│   │   ├── control_server.hpp/.cpp   # gRPC server (controle local/remoto)
│   │   ├── control_client.hpp/.cpp   # gRPC client
│   │   └── proto/                    # Protobuf definitions
│   │       ├── dav_control.proto
│   │       └── dav_metrics.proto
│   │
│   └── metrics/                      # Observabilidade
│       ├── metrics_registry.hpp/.cpp # prometheus-cpp
│       ├── stats_collector.hpp/.cpp  # Coleta de stats por sessão
│       └── http_exporter.hpp/.cpp    # HTTP /metrics endpoint
│
├── tests/
│   ├── unit/
│   │   ├── test_fec.cpp
│   │   ├── test_kcp.cpp
│   │   ├── test_codec.cpp
│   │   ├── test_crypto.cpp
│   │   └── test_jitter_buffer.cpp
│   ├── integration/
│   │   ├── test_lan_stream.cpp       # LAN loopback stream
│   │   └── test_session_reconnect.cpp
│   └── bench/
│       ├── bench_encode.cpp          # Throughput encode
│       ├── bench_kcp.cpp             # Latência KCP vs TCP
│       └── bench_fec.cpp             # Overhead FEC
│
├── scripts/
│   ├── setup_vcpkg.ps1               # Setup vcpkg no Windows
│   ├── setup_vcpkg.sh                # Setup vcpkg no Linux
│   ├── run_stun_server.py            # STUN server de testes
│   └── benchmark_report.py           # Gera relatórios de benchmark
│
└── docs/
    ├── ARCHITECTURE.md               # Este documento expandido
    ├── PROTOCOL.md                   # Especificação do protocolo de wire
    ├── QOS.md                        # Detalhes do motor de QoS
    └── BUILD.md                      # Guia de build
```

Fase 1 — LAN: Distribuição Local de Vídeo

Componentes Ativos

- `capture/dxgi_capturer` (Windows) ou `capture/x11_capturer` (Linux)
- `codec/nvenc_encoder` (GPU) → fallback `sw_encoder`
- `packetizer/rtp_packetizer`
- `transport/kcp_transport` (ARQ sobre UDP)
- `qos/fec` (overhead configurável: 5–20%)
- `session/session_manager`

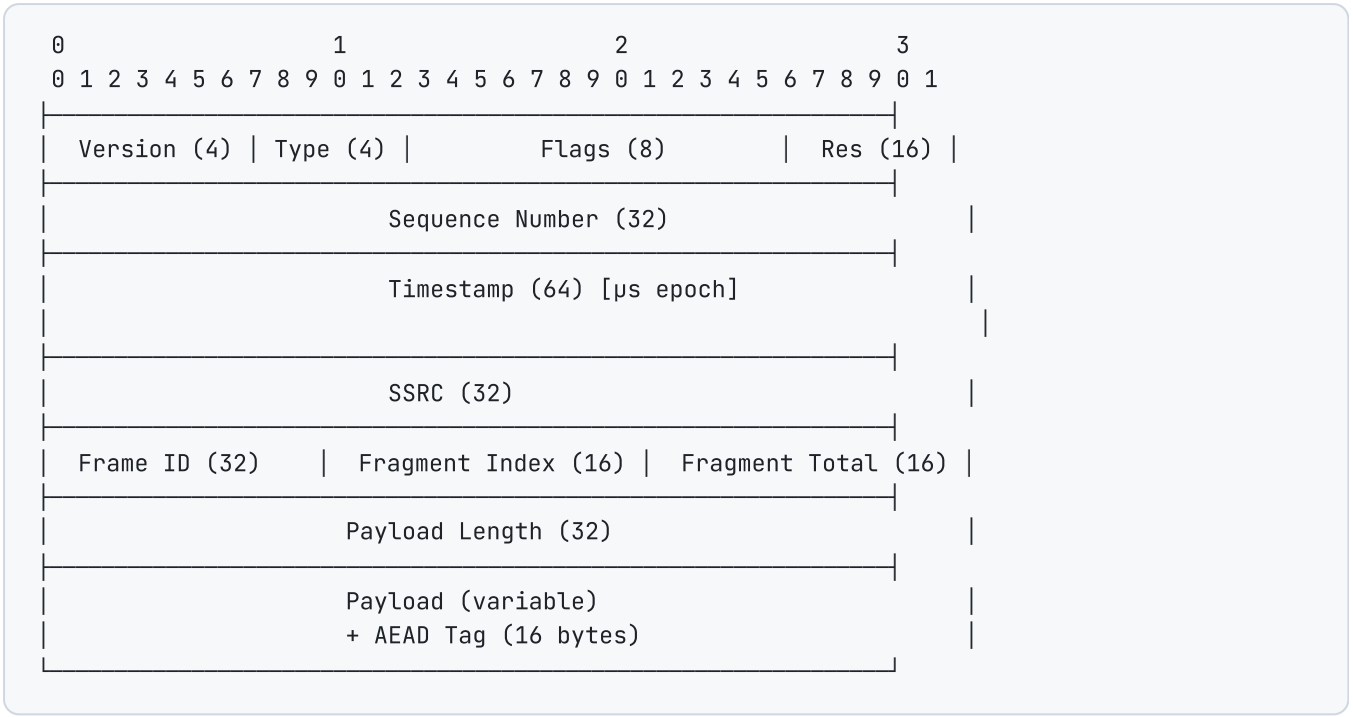
Pipeline de Dados (Sender)

```
[GPU Framebuffer]
      | DXGI AcquireNextFrame (zero-copy)
      ▼
[Raw BGRA/NV12 Frame]  →  frame_buffer (ring, lock-free SPSC)
      | libyuv BGRA→NV12 (GPU, se CUDA disponível)
      ▼
[NV12 Frame]  →  nvenc_encoder  →  H.264/H.265 bitstream
      | slice em NAL units
      ▼
[NAL Units]  →  rtp_packetizer (MTU-aware, max 1400 bytes)
      | adiciona seq_num, timestamp, SSRC
      ▼
[RTP Packets]  →  fec (Reed-Solomon, grupo de N+K)
      |
      ▼
[Packets + FEC]  →  crypto_channel (ChaCha20-Poly1305)
      |
      ▼
[Encrypted Packets]  →  kcp_transport  →  UDP socket
```

Pipeline de Dados (Receiver)

```
[UDP socket]  →  kcp_transport (ARQ, ordering)
      |
      ▼
[Ordered Packets]  →  crypto_channel (decrypt + verify MAC)
      |
      ▼
[Decrypted Packets]  →  fec (recover perdas)  →  jitter_buffer
      |
      ▼
[Ordered RTP]  →  frame_assembler  →  H.264 decoder (ffmpeg)
      |
      ▼
[Decoded Frame]  →  [Consumer / saída configurável]
```

Protocolo de Frame (Wire Format)

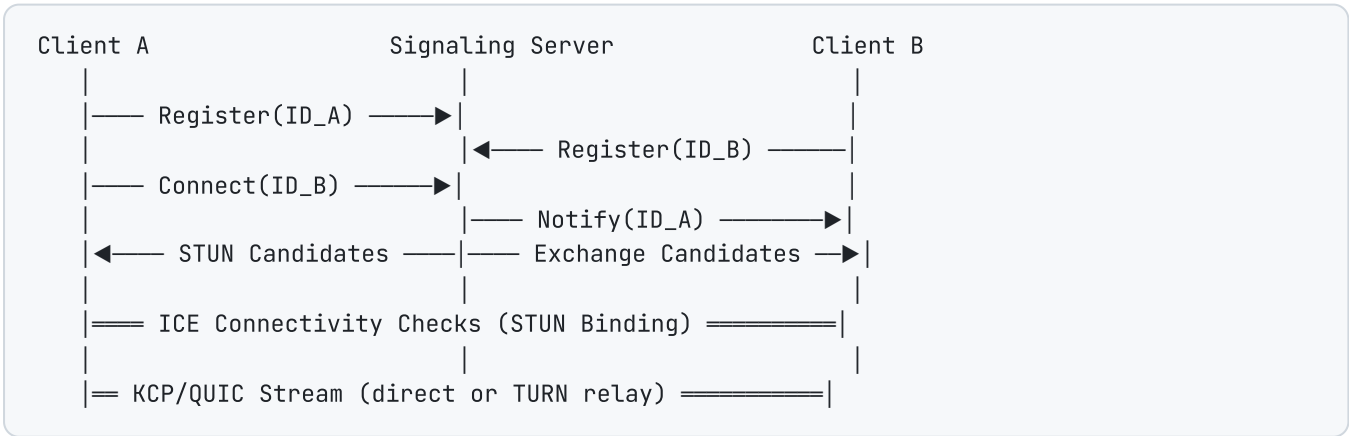


Fase 2 — WAN: Traversal NAT e Comunicação Remota

Componentes Adicionais

- `traversal/ice_agent` — ICE full: host, srflx, relay candidates
- `traversal/stun_client` — STUN RFC 5389
- `traversal/turn_client` — TURN RFC 5766
- `traversal/hole_punch` — UDP hole punching coordenado por servidor de sinalização
- `session/peer_registry` — registro remoto via servidor de rendezvous

Fluxo de Estabelecimento de Conexão WAN



Servidor de Sinalização

- Implementado como serviço leve em **Go** ou **Python asyncio**
- Protocolo: WebSocket + FlatBuffers
- Stateless: não armazena frames, apenas facilita troca de candidatos ICE

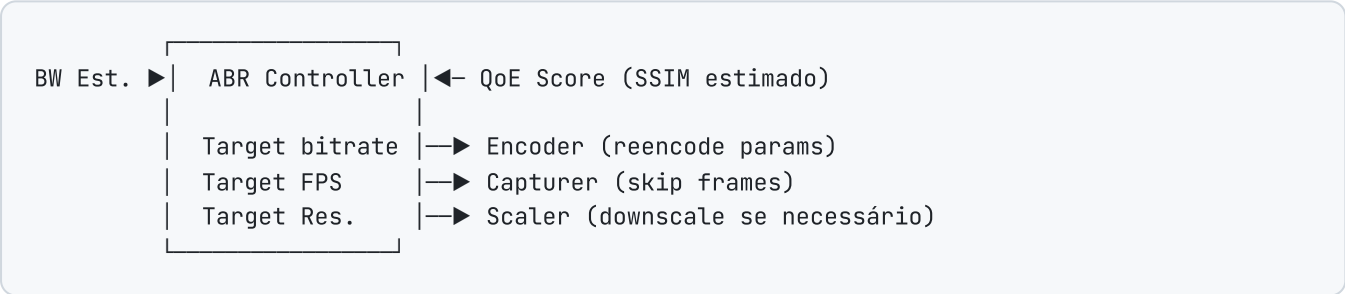
Fase 3 — Full-Mesh + QoS Avançado

Motor de QoS — Detalhes

Estimativa de Largura de Banda

- **BBR v2**: modelo de largura de banda e RTT para controle de envio
- **GCC (Google Congestion Control)**: feedback RTCP-like com delay-based CC
- **Probe packets**: rajadas de sonda para estimar capacidade de uplink

Adaptive Bitrate (ABR)



FEC (Forward Error Correction)

- **Reed-Solomon (n, k)**: configurável. Ex: RS(10,2) = 20% overhead, recupera 2 perdas em grupo de 12
- **XOR FEC leve**: para overhead mínimo em links estáveis
- FEC rate adaptativo: aumenta em links com perda > threshold

NACK e Retransmissão Seletiva

- Receptor envia NACK com bitmask para sequências faltantes
- Sender mantém **reorder buffer** (janela configurável)
- Timeout de retransmissão: min(RTT × 1.5, frame_deadline)

Jitter Buffer Adaptativo

- Target delay = 2× jitter atual (Welford running variance)
- PLC (Packet Loss Concealment): frame freeze ou interpolação bilinear

DSCP / QoS Tagging

- Pacotes de vídeo: DSCP CS4 (video)
- Pacotes de controle: DSCP CS6 (network control)
- Pacotes FEC: DSCP CS3

Multi-Peer / Mesh

- Sessões independentes por peer com QoS isolado
- **SFU-lite embutido**: opção de um nó retransmitir para N receptores sem recodificar (forward encoded frames)

Métricas Expostas

Métrica	Tipo	Descrição
`dav_capture_fps`	Gauge	FPS de captura atual
`dav_encode_fps`	Gauge	FPS de encode
`dav_encode_bitrate_bps`	Gauge	Bitrate de saída do encoder
`dav_transport_rtt_ms`	Histogram	RTT por sessão
`dav_transport_loss_rate`	Gauge	Taxa de perda de pacotes
`dav_fec_overhead_pct`	Gauge	Overhead FEC %
`dav_jitter_ms`	Gauge	Jitter atual
`dav_nack_count_total`	Counter	NACKs enviados
`dav_frames_dropped_total`	Counter	Frames descartados (overload)
`dav_session_uptime_seconds`	Gauge	Uptime por sessão

Requisitos de Desempenho (Targets)

Métrica	LAN (1GbE)	WAN (50Mbps)	WAN Degradado (10Mbps, 2% loss)
Latência ponta-a-ponta	< 10ms	< 80ms	< 150ms
FPS mínimo	60	30	24
Qualidade	1080p H.265	720p H.264	480p H.264
Reconexão (failure)	< 1s	< 3s	< 5s
CPU (encoder side)	< 5% (GPU encode)	< 15%	< 25%

Sequência de Implementação (Sprints)

Sprint 1 — Fundação (Semana 1–2)

- [ ] Scaffold CMake + vcpkg
- [ ] Configuração TOML (`config.hpp`)
- [ ] `UdpSocket` + `KcpTransport` funcional
- [ ] Loopback test: sender→receiver UDP local
- [ ] Logging (spdlog) + estrutura de erros (`std::expected`)

Sprint 2 — Captura e Codec (Semana 3–4)

- [ ] `DxgiCapturer` (Windows) com ring buffer lock-free
- [ ] `NvencEncoder` (H.264 baseline)
- [ ] `SwEncoder` fallback (libx264)
- [ ] `RtpPacketizer` + `FrameAssembler`
- [ ] Benchmark: latência captura→encode→packet

Sprint 3 — Pipeline Completo LAN (Semana 5–6)

- [ ] Integração completa sender pipeline
- [ ] `CryptoChannel` (libsodium, ChaCha20-Poly1305)
- [ ] `KeyExchange` (X25519)
- [ ] `FEC` (Reed-Solomon XOR básico)
- [ ] `JitterBuffer` adaptativo
- [ ] Teste E2E LAN: stream contínuo por 30min

Sprint 4 — Sessão e QoS Base (Semana 7–8)

- [ ] `SessionManager` + `Session` state machine
- [ ] `ReconnectHandler` (exponential backoff)
- [ ] `BandwidthEstimator` (GCC)
- [ ] `AdaptiveBitrate` controller
- [ ] `NackHandler` + reorder buffer
- [ ] Métricas Prometheus + endpoint /metrics

Sprint 5 — WAN e Traversal NAT (Semana 9–11)

- [ ] `StunClient` RFC 5389
- [ ] `IceAgent` (libnice wrapper)
- [ ] `TurnClient` RFC 5766
- [ ] Signaling server (Python asyncio WebSocket)
- [ ] Teste E2E WAN: dois nós em redes diferentes

Sprint 6 — BBR + FEC Adaptativo + Hardening (Semana 12–14)

- [ ] BBR v2 congestion control
- [ ] FEC rate adaptativo
- [ ] DSCP tagging
- [ ] Reed-Solomon FEC completo
- [ ] Stress tests e chaos testing (packetloss artificial)
- [ ] Profiling com perf / VTune

Sprint 7 — Multi-Peer + SFU-lite (Semana 15–16)

- [ ] Multi-sessão isolada com QoS por peer
- [ ] Forward-encoded-frames (SFU-lite)
- [ ] gRPC control plane
- [ ] Documentação final

Open Questions

> ⭐ **Importante**
>
> **Plataforma alvo**: O desenvolvimento principal será em **Windows** (DXGI) com portabilidade Linux planejada, ou Linux-first? Isso impacta a captura e o build system inicial.

> ⭐ **Importante**
>
> **Codec preferido**: Começamos com **H.264** (máxima compatibilidade) ou **H.265** (melhor compressão)? Para WAN em links ruins, AV1 é uma opção futura?

> ⭐ **Importante**
>
> **Protocolo de transporte primário**: **KCP sobre UDP** (menor latência, ARQ próprio) ou **QUIC via libmsquic** (mais robusto, maior complexidade)? Podemos implementar os dois com interface abstrata.

> ⚠️ **Aviso**
>
> **Servidor de sinalização**: Será necessário um servidor público para o modo WAN. Devemos planejar a infraestrutura (VPS simples) ou o DA-V operará só em redes controladas (LAN + VPN)?

> ℹ️ **Nota**
>
> **Build inicial**: Windows com MSVC 2022 ou GCC/Clang? vcpkg resolve a maioria das dependências, mas NVENC requer CUDA Toolkit instalado separadamente.

Verification Plan

Automated Tests

```
# Unit tests
cmake --build build --target run_tests

# Benchmarks
./build/bench/bench_encode --benchmark_format=json
./build/bench/bench_kcp --benchmark_format=json
```

Integration Tests

- Loopback LAN stream 30 min sem drops > 0.1%
- Simulação de perda com `tc netem` (Linux) / `clumsy` (Windows)
- Stress test: 8 sessões simultâneas, CPU < 30%

Manual Verification

- Análise de pacotes com Wireshark (validar FEC, KCP, DSCP)

- Flamegraph com perf/VTune para identificar hot paths
- Comparativo de latência vs. FFmpeg + UDP simples como baseline